

Миграционен план — инфраструктура на МАБИ

Контекст

External developer-ът приключва на **1 август 2026**. Към момента на плана разполагаме с **~2 месеца**. Платформата е **live** с **~350 активни клиента**, а **уебинар се провежда всеки месец** — downtime в уебинарен ден е недопустим.

Мигрират се **3 сървъра** към нова инфраструктура (доставчикът е по наш избор).

Сървър (текущ)	Съдържа	Важни детайли
cPanel (Superhosting)	tonkin.bg — WordPress	Managed, без root , MySQL, ~15GB
Contabo VPS #1	my.tonkin.bg — Next.js	Node.js, env файлове с API ключове, ~2GB
Contabo VPS #2	Video Streamer	100GB+ HLS, PHP + ffmpeg + cron, Zoom webhook incoming.tonkin.bg

Водещи принципи

- Минимален / нулев downtime** — особено около месечния уебинар (нулев толеранс в уебинарен ден).
- Обратимост** — всяка стъпка има rollback; старият сървър остава жив, докато новият не е потвърден.
- Buffer преди 1 август** — приключваме с резерв, докато external dev-ът е още наличен за knowledge transfer и достъпи.
- Тествай преди DNS превключване** — никога „на сляпо“.
- Консолидация на ops модела** — където е разумно, един доставчик/подход, за да е поддръжката проста за екипа, който поема.

1. Целева инфраструктура (за всеки сървър + защо)

Целевият доставчик не е фиксиран, затова аргументирам препоръка. Водещата идея е **консолидация към Hetzner Cloud** (контрол + цена + Docker) + **Vercel за Next.js** (там, където managed услуга е по-силна от self-hosted).

1.1 tonkin.bg (WordPress): cPanel → Hetzner Cloud VPS + Docker

Препоръка: Hetzner Cloud VPS (напр. CPX31/41), WordPress + MariaDB в **Docker Compose**, зад **Nginx Proxy Manager** (автоматичен SSL чрез Let's Encrypt).

Защо:

- cPanel е managed **без root** → ограничава автоматизация, контрол и наблюдаемост.
- Hetzner + Docker дава **root**, **възпроизводимата среда** (Compose файл = инфраструктура като код), по-ниска цена и **общ ops модел** с останалите услуги.
- ~15GB е малко — един CPX31/41 го поема с резерв.

Алтернатива: managed WordPress (Cloudways / Kinsta) — по-малко ops, но премия в цената и без консолидация. Подходящо само ако екипът предпочита изобщо да не управлява сървъри.

1.2 my.tonkin.bg (Next.js): Contabo VPS → Vercel

Препоръка: Vercel (Pro).

Защо:

- Next.js е **роден на Vercel** — нулев ops, автоматичен build/deploy от Git, **preview deployment-и** за всеки PR.
- **Вградено криптирано управление на env променливи** (per environment) — по-сигурно от env файлове на VPS (виж т.4).
- Глобален CDN, авто-скейл. Малкият размер (~2GB) и stateless природата го правят идеален кандидат.

Алтернатива: Hetzner + Docker (ако екипът държи „всичко на едно място“). Възможно е, но за Next.js Vercel печели по простота и сигурност на ключовете.

Зависимост: чете от WP REST API → при миграция на WP трябва да обновим API URL-а (env). Затова редът има значение (т.2).

1.3 Video Streamer: Contabo VPS → Hetzner Cloud VPS + обектно хранилище + CDN

Препоръка:

- **Hetzner Cloud VPS** за обработката (PHP, Node, **ffmpeg**, Whisper, cron, Zoom webhook).
- **100GB+ HLS върху обектно хранилище** (Hetzner Storage Box или S3-съвместимо / Cloudflare R2).
- **CDN** за доставка на видеата (BunnyCDN / Cloudflare).

Защо:

- Обработката изисква **постоянен сървър** (ffmpeg + cron + webhook не са serverless-friendly).
- Но 100GB+ статични файлове **не бива да живеят на скъп VPS диск** — обектно хранилище + CDN е по-евтино, по-надеждно, разтоварва сървъра и прави **бъдещи миграции тривиални** (файловете са отделени от обработващия сървър).
- Zoom webhook (incoming.tonkin.bg) остава на VPS-а.

2. Ред на миграция (кой пръв — най-безопасно)

Принцип: редът върви от **най-нискорисковия и най-обратим** компонент към **най-критичния (данните)** — новата инфраструктура се валидира с по-безопасните миграции, преди да се стигне до необратимите стъпки.

Препоръчан ред:

1 Next.js (my.tonkin.bg) — пръв.

Малък, (почти) stateless, лесно обратим (DNS/CNAME). Vercel preview позволява пълно тестване без риск за продукцията. Продължава да чете от **стария** WordPress, така че миграцията му не засяга останалите услуги. Подходящ като пилотна миграция за валидиране на новата инфраструктура.

1 Video Streamer — втори.

Изолиран, но тежък (100GB) и сложен (cron, webhook). Първоначалният rsync се изпълнява **фоново в продължение на дни**; същинският cutover — в **не-убебителен** прозорец. Изолираността позволява **паралелен пробег** (стар + нов) преди финално превключване.

1 WordPress — последен.

Най-висок blast radius: 15GB данни, 350 потребители, **източник на истината** за Next.js. Правим го последен — вече уверени в новата инфраструктура и с утвърдена zero-downtime процедура. След него обновяваме API URL-а на Next.js (вече на Vercel) към новия WP.

Защо не WP пръв: макар всичко да зависи от него, рискът му е най-голям. „Safest first“ означава да започнем с ниско-рисковото, не с гръбнака.

Около уебинарите: фиксираме месечната уебинарна дата; **никакъв cutover ± 1 ден** около нея; Video cutover-ът задължително далеч от уебинар.

Следват точки 3-7: WordPress zero-downtime, Next.js + сигурни ключове, Video Streamer, топ-3 пуска, timeline.